
Parallel and Distributed Simulation

Parallel Discrete Event Simulation:
Introduction & The Null Message
Synchronization Algorithm

Outline

- Air Traffic Network Example
- Parallel Discrete Event Simulation
 - Logical processes & time stamped messages
 - Local causality constraint and the synchronization problem
- Chandy/Misra/Bryant Null Message Algorithm
 - Ground rules
 - An algorithm that doesn't work
 - Deadlock avoidance using null messages

Event-Oriented Simulation

Event handler procedures

state variables

Integer: InTheAir;
Integer: OnTheGround;
Boolean: RunwayFree;

Arrival Event

{
...
}

Landed Event

{
...
}

Departure Event

{
...
}

Simulation application

Simulation executive

Now = 8:45

Pending Event List (PEL)

9:00

9:16

10:10

Event processing loop

While (simulation not finished)

E = smallest time stamp event in PEL

Remove E from PEL

Now := time stamp of E

call event handler procedure

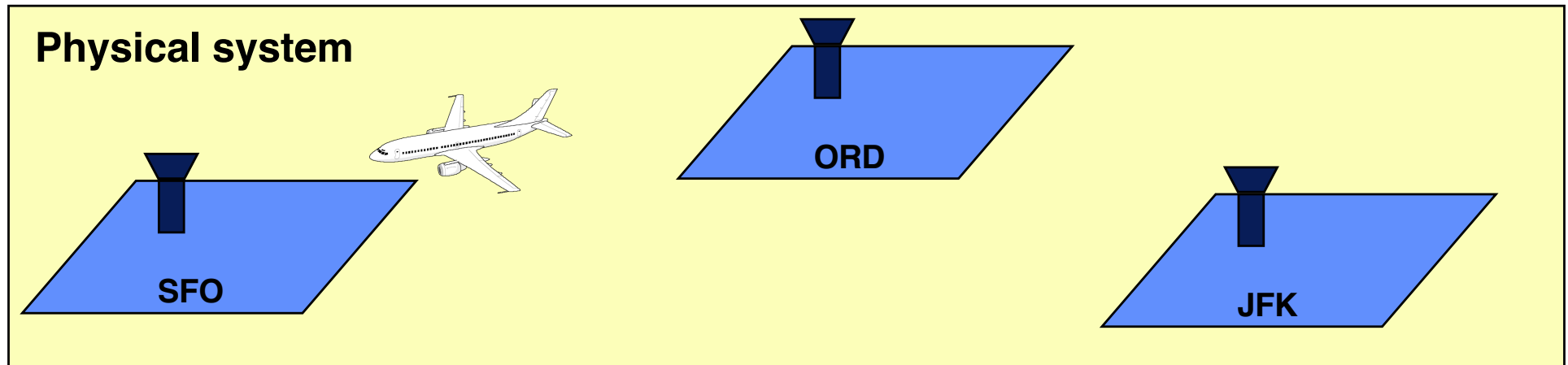
Parallel Discrete Event Simulation

- Extend example to model a network of airports
 - Encapsulate each airport simulator in a **logical process**
 - Logical processes can schedule events (**send messages**) for other logical processes

More generally...

- Physical system
 - Collection of interacting physical processes (airports)
- Simulation
 - Collection of logical processes (LPs)
 - Each LP models a physical process
 - Interactions between physical processes modeled by scheduling events between LPs

Parallel Discrete Event Simulation Example



physical process

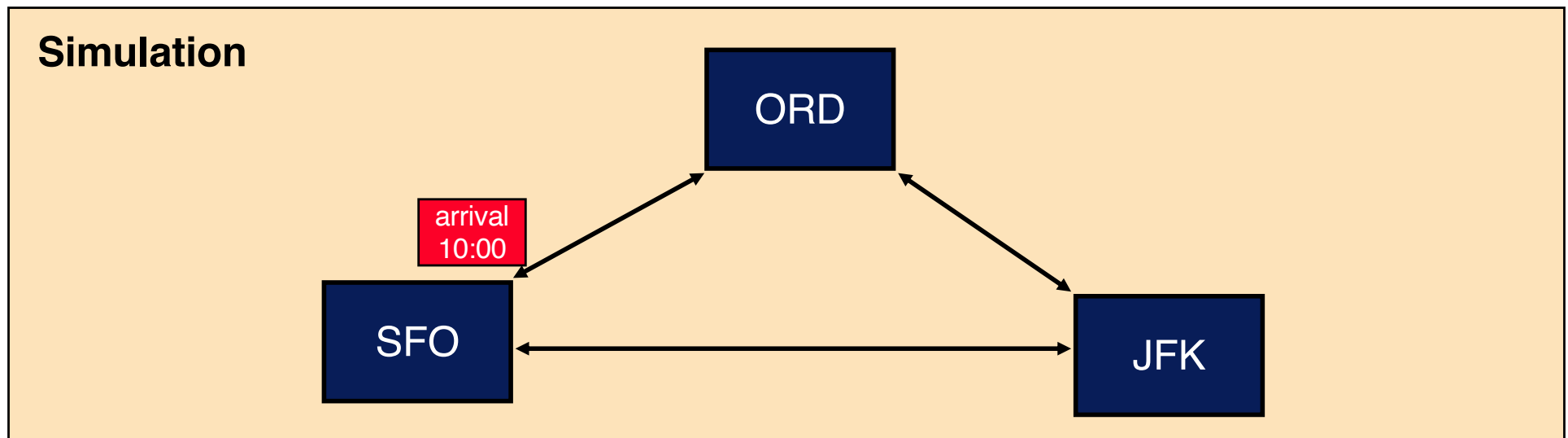


logical process

interactions among physical processes



time stamped event (message)



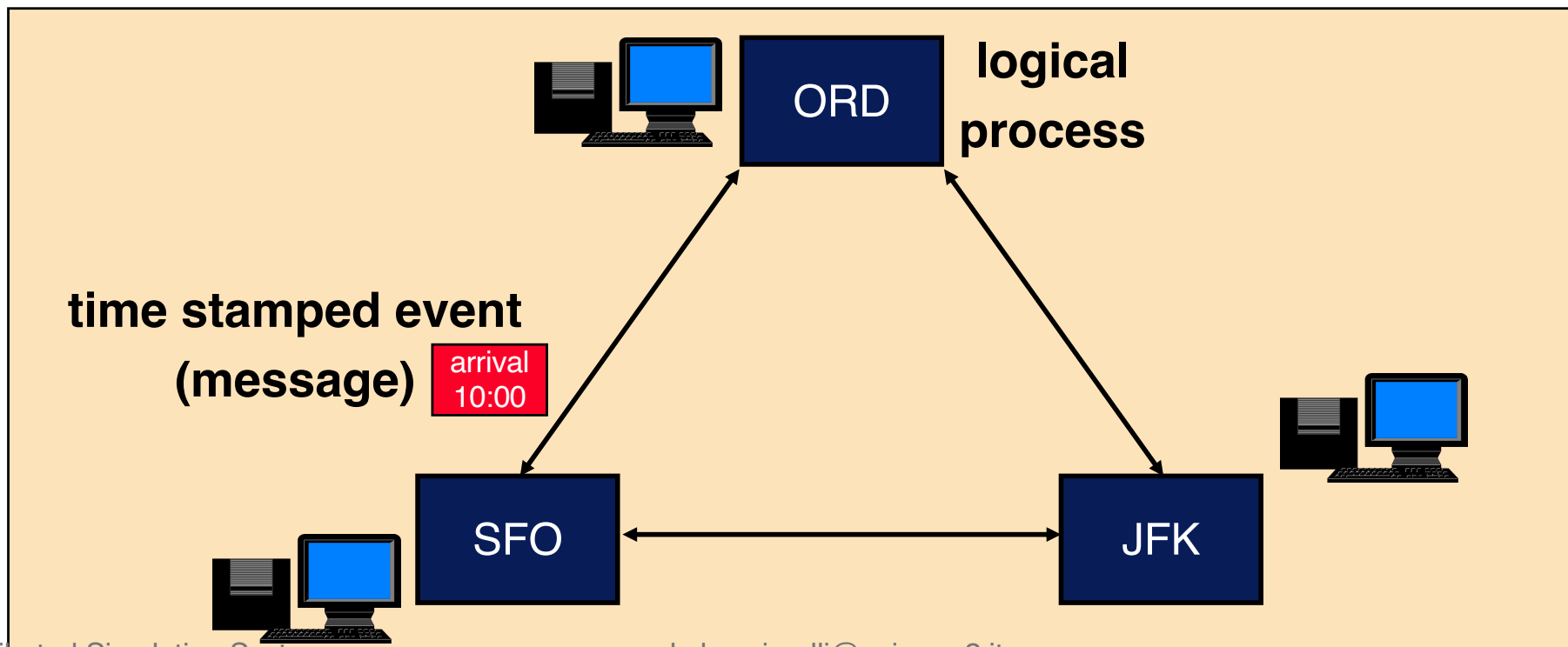
all interactions between LPs must be via messages (no shared state)

LP Simulation Example

- Now: current simulation time
- InTheAir: number of aircraft landing or waiting to land
- OnTheGround: number of landed aircraft
- RunwayFree: Boolean, true if runway available

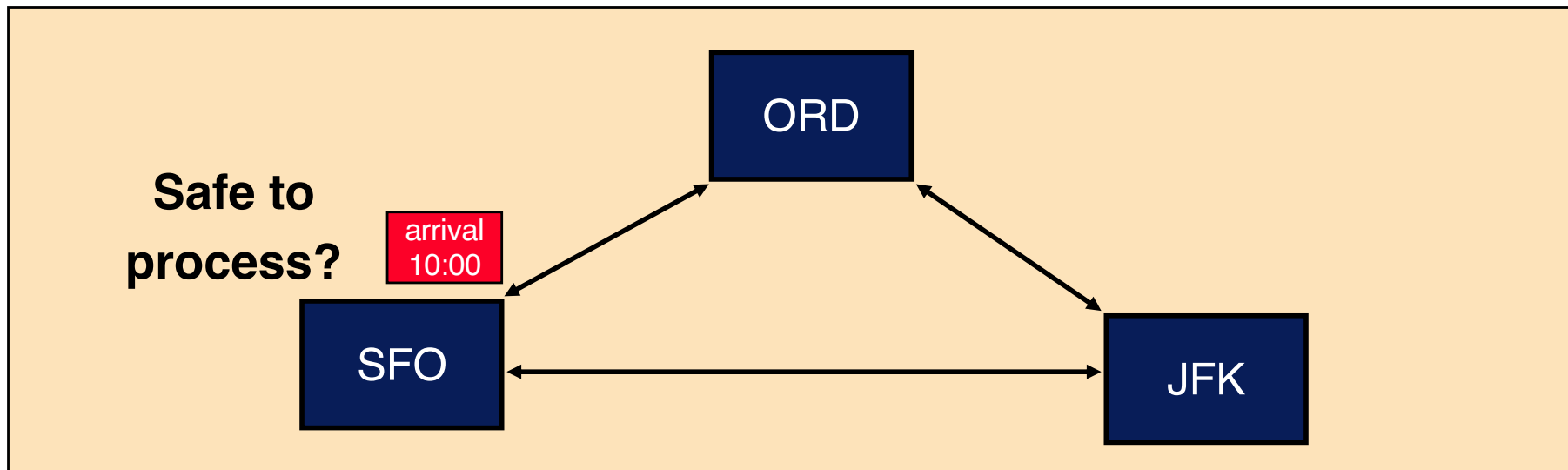
Approach to Parallel/Distributed Execution

- LP paradigm appears well suited to concurrent execution
- Map LPs to different processors
 - Multiple LPs per processor OK
- Communication via message passing
 - All interactions via messages
 - No shared state variables



Causality constraint

Golden rule for each process: “Incoming messages must be processed in time stamp order” (**local causality constraint**)



The Synchronization Problem

Synchronization Problem: An algorithm is needed to ensure each LP processes events in time stamp order

Observation: ignoring events with the same time stamp, adherence to the local causality constraint is sufficient to ensure that the parallel simulation **will produce exactly the same results as a sequential execution** where all events across all LPs are processed in time stamp order.

Synchronization Algorithms

- **Conservative synchronization**: avoid violating the local causality constraint (wait until it's safe)
 - deadlock avoidance using null messages (Chandy/Misra/Bryant)
 - deadlock detection and recovery
 - synchronous algorithms (e.g., execute in “rounds”)
- **Optimistic synchronization**: allow violations of local causality to occur, but detect them at runtime and recover using a rollback mechanism
 - Time Warp (Jefferson)
 - numerous other approaches

Outline

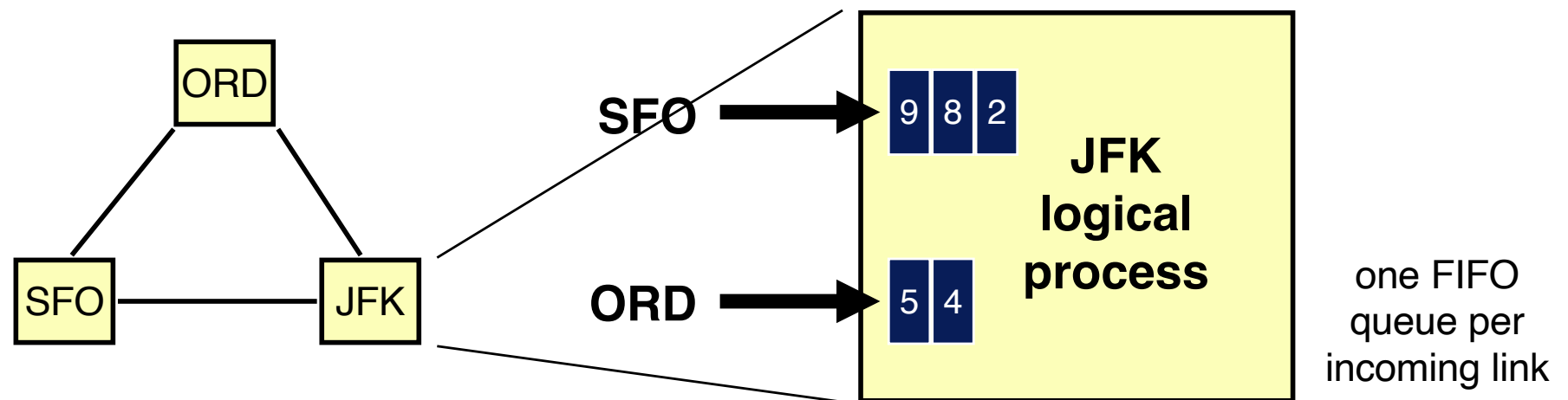
- Air Traffic Network Example
- Parallel Discrete Event Simulation
 - Logical processes
 - Local causality constraint
- Chandy/Misra/Bryant Null Message Algorithm
 - Ground rules
 - An algorithm that doesn't work
 - Deadlock avoidance using null messages

Chandy/Misra/Bryant “Null Message” Algorithm

Assumptions

- logical processes (LPs) exchanging time stamped events (messages)
- static network topology, no dynamic creation of LPs
- messages sent on each link are sent in time stamp order
- network provides reliable delivery, preserves order

Observation: The above assumptions imply the time stamp of the last message received on a link is a **lower bound on the time stamp (LBTS)** of subsequent messages received on that link



Goal: Ensure LP processes events in time stamp order

A Simple Conservative Algorithm

Algorithm A (executed by each LP):

Goal: Ensure events are processed in time stamp order:

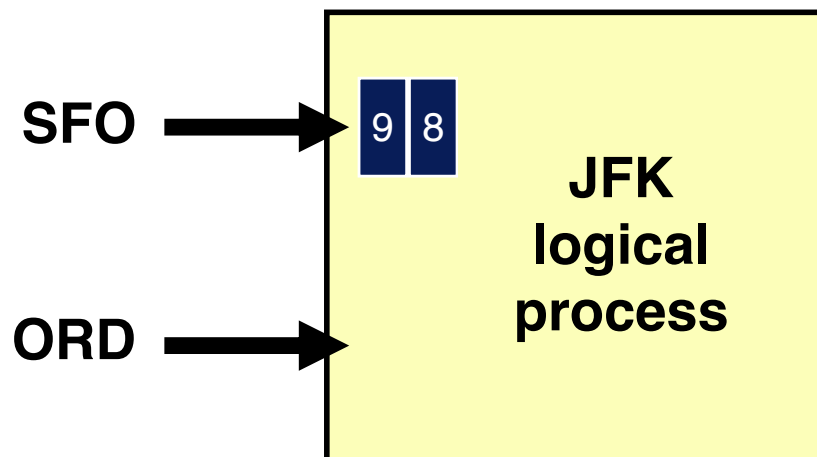
WHILE (simulation is not over)

wait until each FIFO contains at least one message

remove smallest time stamped event from its FIFO

process that event

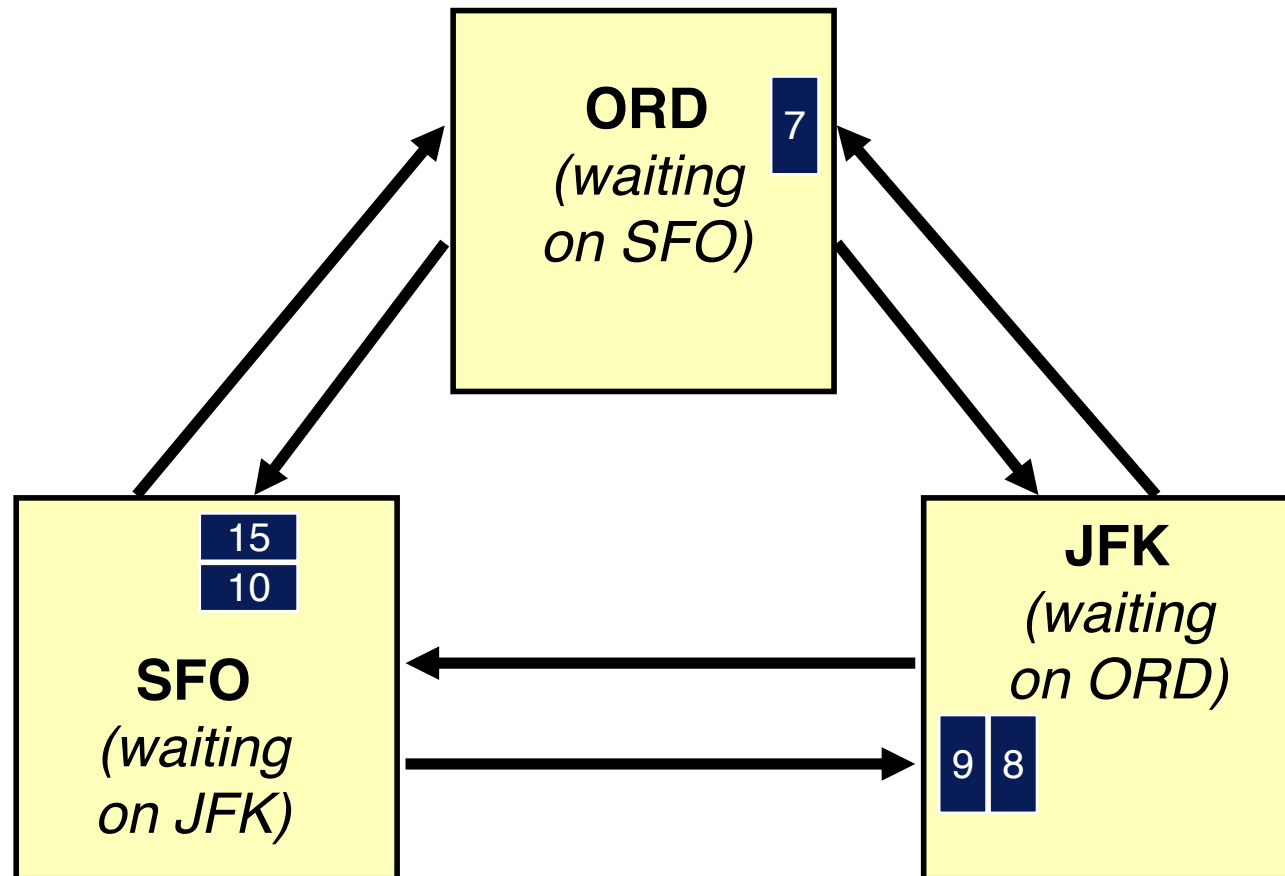
END-LOOP



- process time stamp 2 event
- process time stamp 4 event
- process time stamp 5 event
- wait til message is received from ORD

Observation: Algorithm A is prone to deadlock!

Deadlock Example

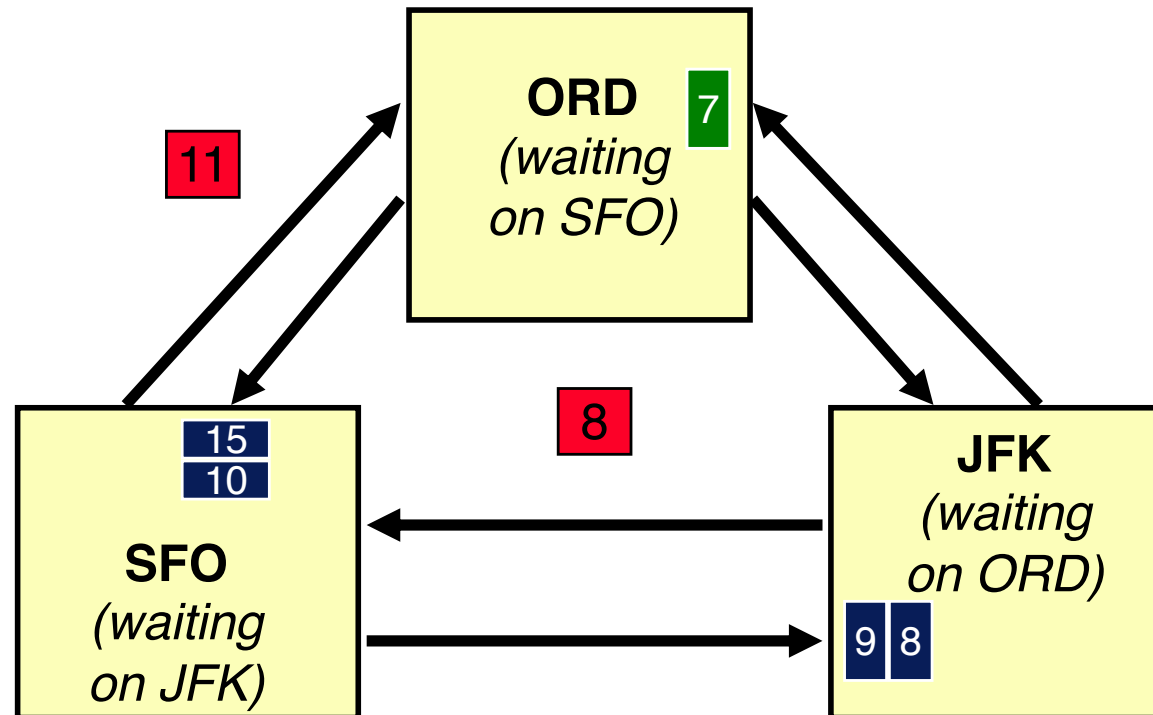


A cycle of LPs forms where each is waiting on the next LP in the cycle.

No LP can advance; the simulation is deadlocked.

Deadlock Avoidance Using Null Messages

Break deadlock: each LP send “null” messages indicating a lower bound on the time stamp of future messages.



Assume minimum delay between airports is 3 units of time

- JFK initially at time 5
- JFK sends null message to SFO with time stamp 8
- SFO sends null message to ORD with time stamp 11
- ORD may now process message with time stamp 7

Deadlock Avoidance Using Null Messages

Null Message Algorithm (executed by each LP):

Goal: Ensure events are processed in time stamp order and avoid deadlock

WHILE (simulation is not over)

wait until each FIFO contains at least one message

remove smallest time stamped event from its FIFO

process that event

send null messages to neighboring LPs with time stamp indicating a lower bound on future messages sent to that LP (current time plus lookahead)

END-LOOP

The null message algorithm relies on a “lookahead” ability.

Summary

- Parallel Discrete Event Simulation
 - Collection of sequential simulators (LPs) possibly running on different processors
 - Logical processes communicating exclusively by exchanging messages
- Chandy/Misra/Bryant Null Message Algorithm
 - Null messages: Lower bound on the time stamp of future messages the LP will send
 - Null messages avoid deadlock

Parallel and Distributed Simulation

Lookahead

Outline

- Null message algorithm: Time Creep
- Lookahead
 - What is it and why is it important?
 - Changing lookahead
 - Deadlock Detection and Recovery Algorithm (Chandy and Misra)

Deadlock Avoidance Using Null Messages

Null Message Algorithm (executed by each LP):

Goal: Ensure events are processed in time stamp order and avoid deadlock

WHILE (simulation is not over)

wait until each FIFO contains at least one message

remove smallest time stamped event from its FIFO

process that event

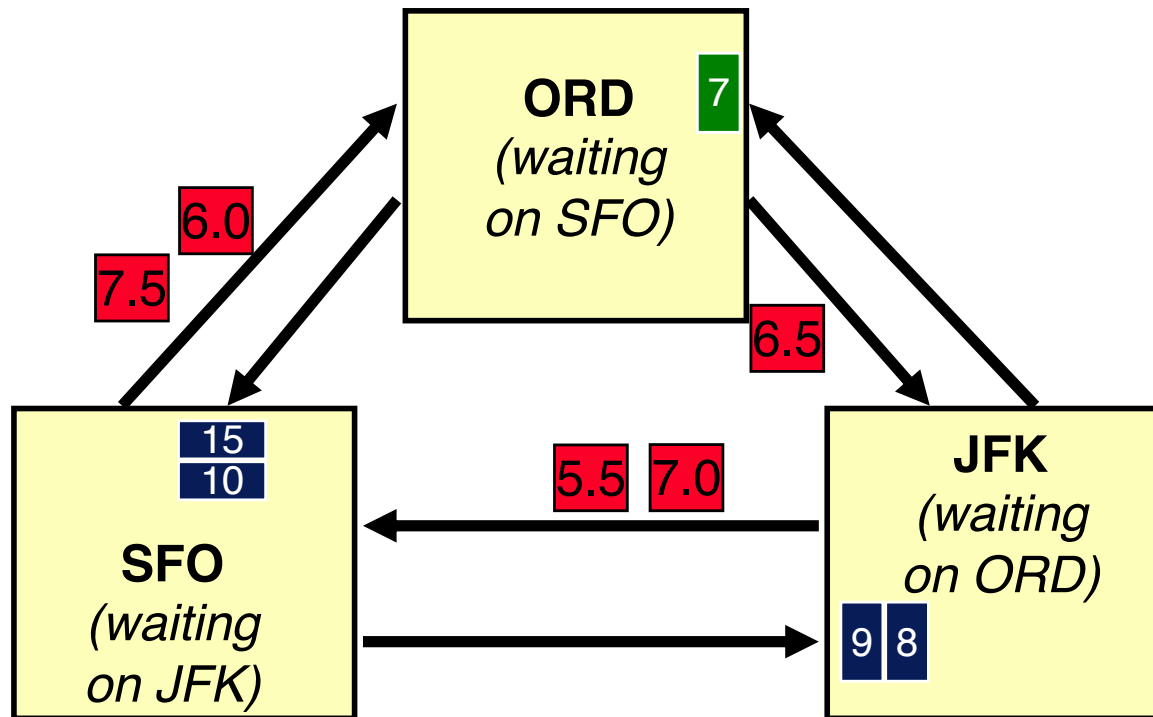
send null messages to neighboring LPs with time stamp indicating a lower bound on future messages sent to that LP (current time plus minimum transit time between airports)

END-LOOP

Variation: LP requests null message when FIFO becomes empty

- Fewer null messages
- Delay to get time stamp information

The Time Creep Problem



Null messages:

JFK: timestamp = 5.5

SFO: timestamp = 6.0

ORD: timestamp = 6.5

JFK: timestamp = 7.0

SFO: timestamp = 7.5

ORD: process time stamp 7 message

Five null messages to process a single event!

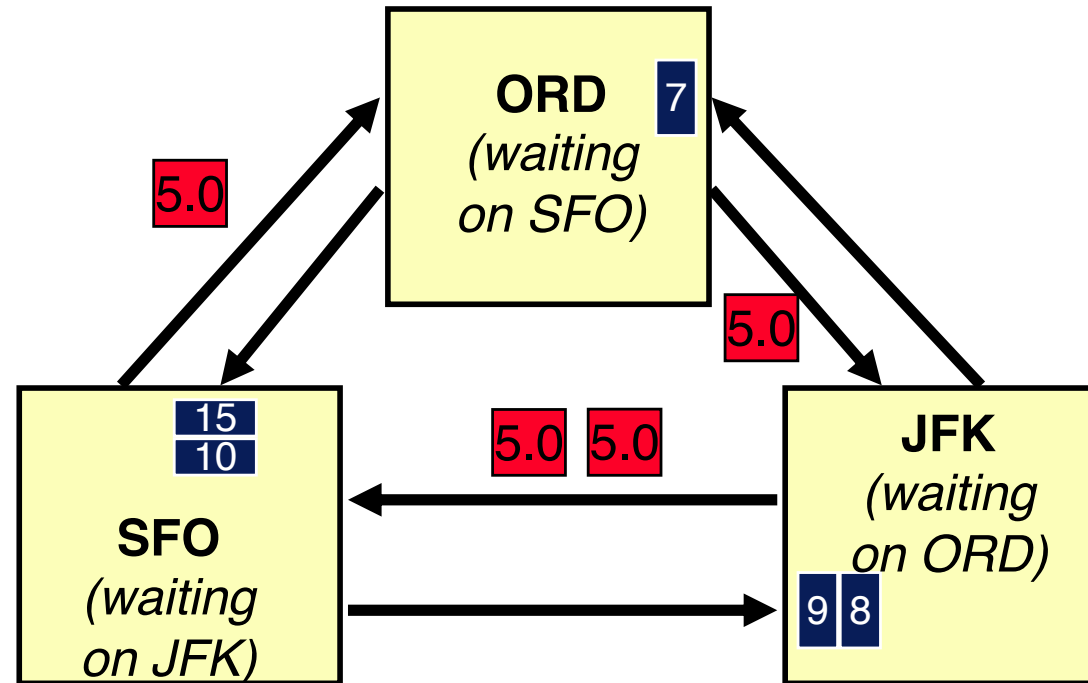
0.5

Assume minimum delay between airports is ~~3~~ units of time
JFK initially at time 5

Many null messages if minimum flight time is small!

Livelock Can Occur!

Suppose the minimum delay between airports is zero!



Livelock: un-ending cycle of null messages where no LP can advance its simulation time

There cannot be a cycle where for each LP in the cycle, an incoming message with time stamp T results in a new message sent to the next LP in the cycle with time stamp T (zero lookahead cycle)

Lookahead

The null message algorithm relies on a “prediction” ability referred to as *lookahead*

- “ORD at simulation time 5, minimum transit time between airports is 3, so the next message sent by ORD must have a time stamp of *at least 8*”

Lookahead is a constraint on LP’s behavior

- **Link lookahead:** If an LP is at simulation time T , and an outgoing link has lookahead L_i , then any message sent on that link must have a time stamp of at least $T+L_i$
- **LP Lookahead:** If an LP is at simulation time T , and has a lookahead of L , then any message sent by that LP must will have a time stamp of at least $T+L$
 - Equivalent to link lookahead where the lookahead on each outgoing link is the same

Lookahead

Lookahead is used to solve this problem:

- Let T_s be the current time of the LP with the ***smallest clock*** value in the entire simulation.
- If each LP has a lookahead of L , then this ***guarantees*** that any new message sent by an LP must have a time stamp of at least $T_s + L$.
- This in turn ***implies*** that all events with time stamp in the interval ***$[T_s, T_s + L]$*** can be safely processed.
- L is referred to as the ***lookahead*** for the LP because it must be able to ***"look ahead"*** L time units into the future and schedule the events at least L time units prior to when they actually happen.